# Structured Label Smoothing over Finite Scalar Quantization for Joint-Embedding Discrete World Models

**Florent Tariolle***
INSA Rouen Normandy

Website Code

## Abstract

Discrete World Models tokenize observations with a learned quantizer and predict next-frame tokens with a transformer, but standard cross-entropy treats every incorrect prediction as equally wrong. Finite Scalar Quantization (FSQ) makes a richer signal available by construction: each code sits on an integer coordinate lattice, so a token one step away in one dimension is a near-miss while a token at the opposite corner is a gross error. We introduce *Structured Label Smoothing* (SLS), which replaces the one-hot training target with a kernel over codebook coordinates, so a near-miss prediction is treated as a near-miss rather than a gross error. We study SLS under joint training of the FSQ encoder and the dynamics transformer in a single discrete latent space, with a straight-through path carrying prediction gradients back into the encoder and a light pixel-reconstruction loss retained only as an anti-collapse regularizer. An isotropic kernel with bandwidth fixed by a first-neighbour rule gives a zero-calibration hyperparameter that is robust to codebook drift. We evaluate on Geometry Dash, where the controller is trained entirely in imagination and deployed at 30 FPS on the real game via screen capture.

## 1 Introduction

Learning WMs that predict future observations from actions is central to model-based reinforcement learning [1]. Recent approaches tokenize visual frames into discrete codes and train autoregressive transformers to predict token sequences [2, 3]. Finite Scalar Quantization [FSQ; 4] offers a simple alternative to VQ-VAE codebooks: each latent vector is independently rounded to one of $L$ levels per dimension, yielding $\prod_d L_d$ implicit codes with no codebook collapse and no auxiliary losses.

A key property of FSQ, largely unexploited in downstream tasks, is that its codebook has a natural coordinate structure. Each code maps to a point on a $D$-dimensional integer lattice, and the distance between two codes in this space is semantically meaningful: codes that differ by one step in one dimension represent visually similar patches. Yet discrete WMs supervise next-token prediction with one-hot cross-entropy on tokenizer indices [2], which exploits no coordinate metric on the latent space and treats a near-miss (one lattice step from the target) and a gross error (opposite corner of the codebook) as equally informative training signals.

We propose *Structured Label Smoothing* (SLS), a soft-target objective whose target distribution is a kernel over FSQ coordinate distances, so the supervision signal reflects the geometry the quantizer has already learned. Classical uniform label smoothing [5] is the structureless limit of this construction and a natural ablation, not the baseline SLS improves over.

This can be viewed as part of a broader shift from data-agnostic heuristics to structure derived from learned representations, analogous to how learned strided convolutions replaced fixed max-pooling

*Correspondence to `florent.tariolle@insa-rouen.fr`

Preprint.

for spatial downsampling [6]. Unlike knowledge distillation, which requires a separate teacher model to produce soft targets, SLS exploits geometry already present in the FSQ codebook at no additional cost.

We study SLS under a joint-training setup in which the FSQ encoder and the transformer are optimized together, so that prediction gradients shape the codebook rather than consuming a fixed tokenization. We evaluate SLS within a complete Vision-Model-Controller (V-M-C) pipeline for Geometry Dash, a fast-paced platformer that demands frame-accurate predictions under a 33 ms latency budget. Our system consists of three components: (1) an FSQ tokenizer that encodes $64 \times 64$ Sobel edge maps into $8 \times 8$ grids of discrete tokens, (2) a causal transformer with Adaptive Layer Normalization (AdaLN-Zero) for action conditioning [7] and QK-norm for attention stability [8], and (3) a lightweight MLP controller trained via PPO entirely in dreamed rollouts [9]. The full pipeline runs in real time at 30 FPS on consumer hardware.

Our contributions are:

1. **Structured Label Smoothing (SLS)**: a training objective that exploits FSQ coordinate geometry to distribute smoothing mass according to codebook distance. We study three kernel families and adopt an isotropic bandwidth set by a first-neighbour rule, giving a zero-calibration hyperparameter that is robust to codebook drift under joint training. The formulation generalizes to any discrete latent space with a coordinate metric.
2. **A real-time WM system**: a complete V-M-C pipeline for Geometry Dash running at 30 FPS, combining joint-trained FSQ tokenization with prediction-gradient feedback, AdaLN-conditioned transformers, and PPO-in-dreams.

## 2 Related work

**Discrete WMs.** The Vision-Model-Controller paradigm [1] introduced encoding frames with a VAE, modeling dynamics with an RNN, and training a controller in dreamed rollouts. IRIS [2] replaced the VAE with a discrete tokenizer and an autoregressive transformer, using an alternating schedule between tokenizer and dynamics updates. DreamerV3 [3] demonstrated that categorical RSSM latents generalize across diverse domains; this approach was scaled further in [10]. TWISTER [11] added action-conditioned contrastive predictive coding to improve temporal coherence. Dreamer-family models do train their encoders jointly with the dynamics model, but use categorical-VAE latents rather than FSQ. FSQ [4] is typically positioned as a frozen tokenizer for downstream tasks, and its coordinate-lattice structure has not been exploited at training time. Our work applies joint training specifically to FSQ so that prediction gradients shape a codebook whose lattice structure SLS can then exploit.

**Finite Scalar Quantization.** FSQ [4] replaces learned codebooks with per-dimension rounding to fixed levels, eliminating codebook collapse and commitment losses. Each code is a point on a $D$-dimensional integer lattice, giving the codebook a natural coordinate structure. iFSQ [12] introduces a simple modification of the FSQ bounding function that improves bin utilization; we adopt it in our encoder.

**Joint-Embedding Predictive Architectures (JEPA).** LeWorldModel [13] recently demonstrated a stable end-to-end JEPA WM from pixels using continuous latents and a SIGReg anti-collapse regularizer. SLS-WM is joint-embedding in the architectural sense that context and target frames are encoded by a single online FSQ encoder into a shared discrete latent space, and the transformer predicts future tokens $z_{t+1}$ from past tokens $z_{\leq t}$ in that space, with no pixel-prediction head. It departs from strict JEPA in two respects: the embedding space is discrete rather than continuous, and the predictive loss is a cross-entropy over FSQ indices rather than a distance between continuous embeddings – SLS can be read as narrowing this gap, by replacing uniform CE with a lattice-metric-weighted CE that reflects embedding distance on the codebook. The pixel decoder is retained only as an anti-collapse regularizer on the encoder, via a light pixel-reconstruction loss that plays no role in the predictive loop; removing it causes the encoder to collapse under joint training. We attempted to replace the decoder by adapting SIGReg to the discrete factorized FSQ case via a Cramér-von-Mises test against a factorized uniform target (a marginal anti-collapse signal in the spirit of [14]); in our experiments this was insufficient to prevent joint-distribution degeneracy when the prediction target

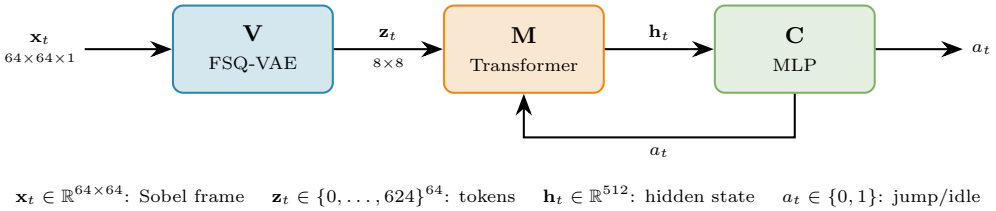


Figure 1: Vision-Model-Controller pipeline. The FSQ-VAE (V) tokenizes Sobel edge maps into a grid of discrete codes, the causal transformer (M) predicts next-frame tokens conditioned on actions via AdaLN, and the MLP controller (C) selects actions from the transformer's hidden state. The action feedback loop enables autoregressive dreaming.

comes from the online encoder, and finding a decoder-free discrete anti-collapse signal is left as open work.

**Label smoothing.** Label smoothing [5] proposed replacing one-hot targets with a uniform mixture to reduce overconfidence. While effective as a regularizer, uniform smoothing ignores class relationships. Knowledge distillation and soft-target methods distribute mass based on teacher predictions, but require a separate teacher model. We instead derive the target distribution from the codebook geometry itself, requiring no additional model.

**Adaptive conditioning.** FiLM [15] introduced adaptive affine conditioning via scale and shift. DiT [7] applied this as AdaLN-Zero with zero-initialized gates for stable training of diffusion transformers. QK-norm [8] (RMSNorm on queries and keys per head) was introduced to stabilize attention logits in deep AdaLN models. LeWorldModel [13] applied AdaLN to world-model transformers. We adopt AdaLN-Zero with QK-norm for action conditioning, finding that QK-norm is essential for training stability (Section 3.4).

## 3 Method

Our system follows the Vision-Model-Controller (V-M-C) paradigm [1] (Figure 1). We describe each component below, with Structured Label Smoothing (Section 3.3) as the primary contribution, and joint training of the FSQ encoder with the transformer (Section 3.2) as the setup in which SLS is studied.

### 3.1 FSQ tokenization (Vision)

We encode $64 \times 64$ grayscale Sobel edge maps using an FSQ-VAE with levels $[5, 5, 5, 5]$, producing $625$ implicit codes. The encoder applies three stride-2 convolutional stages with residual blocks, projecting to a 4-channel latent map of size $8 \times 8$. Each spatial position is independently quantized using the iFSQ bounding function [12], $2\sigma(1.6z) - 1$ in place of $\tanh(z)$, scaled to half-levels and rounded, yielding $64$ tokens per frame. The decoder mirrors the encoder with transposed convolutions and is retained during joint training as a pixel-space anchor (Section 3.2).

### 3.2 Joint training of FSQ and transformer

FSQ was originally introduced as a tokenizer for downstream use and is typically frozen once trained [4]. Under a frozen tokenizer, the cross-entropy loss only shapes the predictor; the codebook geometry is determined purely by reconstruction. We instead train the encoder jointly with the transformer so that prediction gradients also shape the codebook.

**Gradient routing with a straight-through estimator.** The transformer consumes discrete token indices via an embedding table, so prediction gradients cannot flow through rounding in the usual way. We insert a learnable linear projection `fsq_grad_proj` between the encoder's continuous latents

(pre-rounding) and the transformer's input representation. The projection contributes zero in the forward path (the transformer still reads the discrete indices) but, under a straight-through estimator, carries the prediction gradient back into the encoder's continuous features. The encoder therefore receives two gradient signals: next-token prediction through the straight-through path, which shapes the codebook for the predictive task, and pixel reconstruction through the decoder, which acts as an anti-collapse regularizer (discussed below).

**Anti-collapse regularization on the encoder.** Prediction happens entirely in the FSQ latent space, so the decoder plays no role in the predictive loop. Without an external signal, however, joint training of a single online encoder with its own prediction target admits a trivial degenerate solution: the encoder maps every input to the same point and the transformer predicts that point with zero loss. We prevent this by keeping a light pixel-MSE reconstruction loss on the encoder, which is therefore used as an anti-collapse regularizer rather than as a predictive objective. Our adaptation of SIGReg [13] to the discrete factorized FSQ case, using a Cramér-von-Mises test against a factorized uniform target, was insufficient to prevent joint-distribution degeneracy in our setting; finding a decoder-free anti-collapse signal strong enough to remove the pixel-MSE crutch is left as future work.

**Single-group optimization and global gradient clipping.** Under joint training the pixel-MSE and cross-entropy gradients live on different scales, but we do not separate them into different parameter groups. Following the joint-training recipe of [13], the encoder and transformer sit in a single AdamW group with a shared learning rate and weight decay, and a single global gradient-norm clip is applied across the combined parameter set. The clip threshold is set large enough (empirically $\sim 10\times$ the transformer gradient norm at convergence) that the reconstruction signal is preserved while residual spikes on either side are still caught. Separating the two into per-group clips with a tight threshold, as in our earlier iterations, consistently produced a positive train-validation gap on the same budget; single-group with a wide clip yielded smoother training curves and closes that gap.

**No encoder freeze.** We also tried freezing the encoder at a pre-set fraction of training (motivated by the observation that FSQ converges faster than the transformer in two-stage setups). In practice this removed a source of slow codebook adaptation that the transformer continued to exploit late in training, and the resulting runs did not dominate their unfrozen counterparts on any metric we tracked. We therefore keep the encoder unfrozen for the full schedule, and the pixel-MSE anchor stays active throughout.

### 3.3 Structured Label Smoothing

**Motivation.** Standard label smoothing [5] replaces a one-hot target $\mathbf{e}_i$ with

$$q_{\text{uniform}}(j \mid i) = (1-\varepsilon)\,\delta_{ij} + \frac{\varepsilon}{V-1}(1-\delta_{ij}), \tag{1}$$

where $V = 625$ is the vocabulary size and $\varepsilon$ controls the smoothing strength. This distributes $\varepsilon$ uniformly across all alternatives, ignoring codebook geometry.

**FSQ coordinate distance.** An FSQ codebook with levels $\mathbf{L} = (L_1, \ldots, L_D)$ assigns each code $i$ a coordinate vector $\mathbf{c}(i) = (c_1(i), \ldots, c_D(i))$ on a $D$-dimensional integer lattice. We define the squared lattice distance

$$d^2(i,j) = \sum_{d=1}^{D} \left(c_d(i) - c_d(j)\right)^2. \tag{2}$$

**Kernel-family study.** We compare three kernels over $d = \sqrt{d^2(i,j)}$:

$$k_{\text{Gauss}}(d) = e^{-d^2/2\sigma^2}, \qquad k_{\text{Laplace}}(d) = e^{-d/\sigma}, \qquad k_{\text{Cauchy}}(d) = \frac{1}{1 + d^2/\sigma^2}. \tag{3}$$

The three families differ in tail behaviour. Cauchy has polynomial tails and leaks meaningful mass to distant neighbours; Laplace has a cusp at $d = 0$ and an exponential tail; Gaussian has the thinnest tails, decaying faster than either alternative once $d$ exceeds the bandwidth. We take Gaussian with $\sigma = 0.7$ as the current default: thin tails ensure distant neighbours receive negligible mass ($k(2) \approx 0.02$), while the bandwidth still places meaningful mass on the immediate lattice neighbour ($k(1) \approx 0.37$, matching Laplace at $\sigma = 1$).

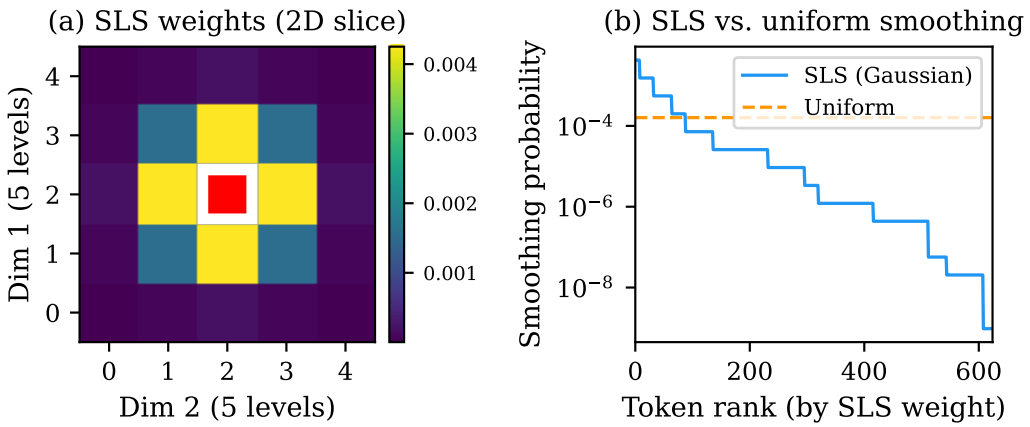


Figure 2: SLS target distribution for a token near the centre of the codebook. **(a)** 2D slice over two lattice dimensions with the remaining two held at the target's values: weights decay as a Gaussian kernel over lattice distance, red square marks the target. **(b)** All $V-1$ alternative tokens sorted by SLS weight (log scale) vs. uniform label smoothing. SLS concentrates mass on near-miss tokens while suppressing distant ones.

**Isotropic first-neighbour bandwidth.** Prior to joint training we calibrated per-dimension weights $\alpha_d$ from an empirical sensitivity probe: perturbing each FSQ dimension by $\pm 1$, decoding, and measuring reconstruction MSE. Under joint training the codebook drifts continuously, so per-dimension weights calibrated from any single snapshot go stale. We instead use an isotropic kernel and fix $\sigma$ by a geometric rule: choose the bandwidth so that the immediate integer lattice neighbour sits near the kernel's half-maximum. This gives a zero-calibration hyperparameter and is robust to codebook drift. The sensitivity probe is retained as an analysis tool to characterize a learned codebook's anisotropy and cross-dimension coupling, but not as a hyperparameter source.

**Structured smooth target.** We replace the uniform distribution in Equation 1 with a kernel over FSQ distances:

$$q_{\text{SLS}}(j \mid i) = \begin{cases} 1-\varepsilon & \text{if } j = i, \\ \varepsilon \cdot \dfrac{k(d(i,j))}{\sum_{l \neq i} k(d(i,l))} & \text{otherwise.} \end{cases} \tag{4}$$

The full target matrix $\mathbf{Q} \in \mathbb{R}^{V \times V}$ is precomputed once per kernel choice and stored on GPU (Figure 2). We use $\varepsilon = 0.1$.

**Integration with focal loss.** To handle class imbalance (background tokens dominate over obstacle and spike tokens), we combine SLS with focal loss [16]:

$$\mathcal{L}_{\text{SLS-focal}} = -\sum_{j=1}^{V} (1-p_j)^{\gamma}\, q_{\text{SLS}}(j \mid i)\, \log p_j, \tag{5}$$

where $p_j = \text{softmax}(\mathbf{z})_j$ and $\gamma = 2.0$.

### 3.4 Transformer WM (Model)

The WM is a causal transformer (512 embedding dim, 8 heads, 8 layers) that predicts the next frame's tokens given a context of $K = 4$ past frames and their associated actions.

**Sequence construction.** Each frame contributes 64 visual tokens (from the $8 \times 8$ FSQ grid) plus one status token (ALIVE or DEATH), for 65 tokens per frame. A context window of $K+1$ frames (including the target) produces a sequence of $(K+1) \times 65 = 325$ tokens. All target-frame tokens are replaced with a learnable `[MASK]` embedding to prevent information leakage.

**3D Rotary Position Embedding.** We apply factored RoPE [17] encoding three positional axes: spatial row, spatial column, and temporal frame index. The head dimension is partitioned into three bands (rows, columns, time) with separate base frequencies ($\theta_{\text{spatial}} = 10$, $\theta_{\text{time}} = 10{,}000$). Status tokens receive the grid centre coordinates to represent whole-frame summaries.

**Block-causal attention.** Within each frame block, attention is bidirectional (tokens attend to all positions in the same frame). Across frames, attention is strictly causal: each frame attends only to current and past frames.

**AdaLN-Zero action conditioning.** Rather than inserting action tokens into the sequence, we condition the transformer on the action history via Adaptive Layer Normalization [7, 15]. The $K$-step action sequence is one-hot encoded and projected through an MLP to produce per-layer scale, shift, and gate parameters that modulate each sublayer:

$$\text{AdaLN}(\mathbf{x}) = (1 + \mathbf{s}) \odot \text{LayerNorm}(\mathbf{x}) + \mathbf{b}, \qquad \mathbf{y} = \mathbf{g} \odot f(\text{AdaLN}(\mathbf{x})), \tag{6}$$

where $f$ is the attention or MLP sublayer and $(\mathbf{s}, \mathbf{b}, \mathbf{g})$ are predicted from the action context. Unlike standard AdaLN-Zero [7], which initializes gates to zero, we set gate bias $= 1$ to preserve residual flow at initialization; zero-initialized gates suppress both conditioning and learning in this setting (Section 5.1).

**QK-norm.** We apply per-head RMSNorm to queries and keys before computing attention scores [8]. This prevents attention logit overflow that otherwise causes training divergence in AdaLN-conditioned models (Section 5.1).

**FSQ-neighbor token noise.** On context tokens, with probability 5% per token we replace the token with a lattice neighbour obtained by perturbing a single FSQ dimension by $\pm 1$. An earlier dual-noise scheme also applied uniform random-token replacement, but uniform replacement contradicts the SLS geometry (it presents distant tokens as substitutable for the target, the opposite of the SLS signal on the loss side). We kept only the FSQ-neighbor branch so that context and loss agree on what counts as a near-miss.

**Output projection and total loss.** The output projection shares weights with the input embedding (weight tying). The total training loss is

$$\mathcal{L} = \mathcal{L}_{\text{SLS-focal}} + \lambda_{\text{recon}} \mathcal{L}_{\text{MSE}}, \tag{7}$$

applied to all parameters (encoder + transformer) throughout training.

### 3.5 PPO controller (Controller)

The controller is a lightweight MLP that takes the transformer's hidden state $\mathbf{h}_t$ as input and outputs a jump probability and a state value estimate.

**Architecture.** A single-hidden-layer MLP (512 hidden units with LayerNorm and SiLU) is followed by zero-initialized linear actor and critic heads. Input is the transformer's hidden state $\mathbf{h}_t \in \mathbb{R}^{512}$; output is a jump probability and a state value. The full controller has $\sim$265K parameters.

This design follows the TWISTER [11] and DreamerV3 [3] pattern where the controller operates on the latent state produced by a trained WM. Our $\mathbf{h}_t$ already encodes spatial and temporal structure through 8 transformer layers with 3D-RoPE and AdaLN action conditioning, so an additional encoder on the token grid would duplicate work done upstream.

**Training in dreams.** The controller is first pretrained via Behavioural Cloning (BC) on expert demonstrations, then fine-tuned with PPO [9] in dreamed rollouts. At each iteration, 4096 rollouts of up to 25 steps are generated by autoregressively sampling from the frozen WM. Rollouts terminate when the predicted death probability exceeds 0.5. Rewards are $+1$ per survival step with a $-0.25$ penalty per jump to discourage gratuitous jumping (an idle agent dies immediately upon reaching an obstacle, while a jumping agent survives longer by spending time airborne, creating a deceptive local optimum). We use GAE [9] with $\gamma = 0.995$, $\lambda = 0.95$, and clip ratio $\varepsilon_{\text{PPO}} = 0.2$. A Dreamer-style $\lambda$-return actor-critic is a natural replacement: on-policy imagined rollouts make PPO's off-policy clipped surrogate unnecessary. We leave this to future work.

## 4 Experimental setup

**Environment.** Geometry Dash is a rhythm-based platformer where the player controls a cube moving at constant horizontal speed. The only action is a binary jump. Colliding with any obstacle causes instant death. Levels are deterministic: the same action sequence always produces the same outcome, but frame-accurate timing is required.

**Data collection.** We collect roughly 4,200 gameplay episodes ($\sim$250K frames) including both intentional-death episodes (covering diverse failure modes) and 36 expert demonstrations (full level clears). Vertical shift augmentation ($\pm 2, \pm 4$ pixels) expands the dataset $5\times$. Frames are preprocessed with Sobel edge detection and resized to $64 \times 64$.

**Training details.** The FSQ-VAE and transformer are trained jointly with AdamW using cosine annealing and warmup. Both subnetworks sit in a single optimizer group sharing one learning rate ($10^{-3}$) and one weight decay ($10^{-3}$), following the recipe of [13]; earlier two-group configurations (separate encoder and transformer learning rates, larger weight decay on the transformer) produced noisier training curves and a positive train-validation gap on the same budget. We apply 5% FSQ-neighbor token noise on context tokens. Death frames are oversampled $4\times$ via weighted sampling. BF16 mixed-precision training is used throughout; BF16 requires no loss scaling and is numerically more stable than FP16. A single global gradient-norm clip at $10.0$ is applied to the combined parameter set; this value is sized to the empirical encoder gradient scale under the recon-dominated joint objective, so the pixel-MSE gradient does not crowd out the prediction side while residual spikes are still caught. Training runs for 200 epochs on a single NVIDIA A100 GPU; the FSQ encoder is updated throughout, jointly with the transformer (an earlier halfway-freeze schedule was explored but abandoned as it removes a source of slow codebook adaptation that remains informative to the transformer late in training).

**Training throughput.** We benchmarked `torch.compile` modes under BF16 on our joint-training workload using subprocess isolation to prevent CUDA allocator fragmentation across runs. All `torch.compile` modes are within a few percent of each other; we adopt `reduce-overhead` with `fullgraph=True` as the default: it is faster than `default` mode while avoiding the lengthy autotune phase that `max-autotune` adds at startup in exchange for marginal additional steady-state throughput. This configuration is applied to the world-model and controller training scripts. A secondary NVIDIA RTX 2060 SUPER (Turing, SM 7.5) was used in parallel for auxiliary experiments. Turing lacks native BF16 (software-emulated and much slower than FP32 on this card) and has no TF32 support; Inductor's channels-last layout heuristic additionally triggers a stride assertion in the convolution backward, which we disable by setting `torch._inductor.config.layout_optimization = False`. Under these constraints the best local configuration is `torch.compile` in `default` mode with FP16 and a `GradScaler`.

**Real-time inference.** Deployment must process each frame within a 33 ms budget to sustain 30 FPS in the live game. We overlap screen capture with GPU inference: each captured frame is staged into a pinned-memory tensor and uploaded to the GPU with `non_blocking=True`, so the next CPU-side capture can issue while the previous frame's encode-and-predict pass is still executing. The transformer's context window is held as a GPU-resident ring buffer of FSQ token indices, eliminating a GPU$\rightarrow$CPU$\rightarrow$GPU round-trip per frame. The encoder forward pass is wrapped in a CUDA Graph after a short warm-up, and the entire model graph is compiled with `torch.compile` in `reduce-overhead` mode. Under these optimizations the full pipeline (screen capture, Sobel preprocessing, FSQ encode, transformer step, controller action) sustains 30 FPS on a single RTX 2060 SUPER.

**BC pretraining ablation.** To verify that BC pretraining is a convergence accelerator rather than a stability requirement, we train one PPO run from random initialization. Cold-start PPO converges to the same plateau as BC-initialized PPO on all metrics (eval survival, entropy, jump ratio) but requires roughly 2,000 additional iterations ($\approx$6 hours on A100); BC itself takes $\sim$5 minutes, giving a large compute ROI. BC is retained in the pipeline on this basis.

**Evaluation metrics.** We evaluate on: (1) next-token prediction accuracy and cross-entropy on held-out episodes, (2) death prediction F1 score, and (3) real-environment gameplay performance measured as percentage of level completed (mean over $N = 100$ automated runs).

# 5 Results

*Results pending model freeze and ablation experiments.*

## 5.1 Training stability of AdaLN variants

During development of an earlier (frozen-tokenizer) version of this architecture, we ran five AdaLN configurations to identify the minimal set of stabilization techniques required. The specific numerical traces below are retrospective observations from that V4-era line and are reported to motivate the QK-norm choice carried into the current joint-training architecture.

1. **No gates**: diverged mid-training from attention logit explosion.
2. **Zero-init gates**: plateaued early with overfitting; zero-initialized gates suppress residual flow, killing both conditioning and learning.
3. **Gate bias** $= 1$: restored learning speed but attention logits remained unbounded and the run diverged.
4. **Tanh bounding on modulation** $+$ **gate bias** $= 1$: still diverged, demonstrating that the explosion source was not in the modulation parameters.
5. **QK-norm** $+$ **gate bias** $= 1$: stable for full-length training, surpassing all previous configurations. Per-head RMSNorm on queries and keys eliminated the attention logit overflow that was the root cause.

This progression demonstrates that in AdaLN-conditioned WM transformers, attention stability (via QK-norm) is more critical than modulation bounding.

# 6 Limitations

Our evaluation is limited to a single game (Geometry Dash) with a binary action space and deterministic level layouts. The generalization of SLS to environments with higher-dimensional actions, stochastic dynamics, or RGB observations remains to be validated. The pixel decoder is retained only as an anti-collapse regularizer on the encoder: prediction is otherwise entirely in FSQ latent space, and the decoder plays no role at inference time. This is a practical solution, not a principled one: finding a decoder-free discrete anti-collapse signal (our adaptation of SIGReg to the factorized-uniform case via a Cramér-von-Mises test was insufficient against joint-distribution degeneracy) is open work.

# 7 Conclusion

We introduced Structured Label Smoothing, a training objective that exploits the coordinate geometry of FSQ codebooks, and studied it under a joint-training setup in which prediction gradients shape the codebook through a straight-through estimator with a pixel-reconstruction anchor. A kernel-family study over Gaussian, Laplace, and Cauchy families, combined with an isotropic first-neighbour bandwidth, gives a zero-calibration hyperparameter that is robust to the codebook drift induced by joint training. The resulting system is trained end-to-end, with a single AdamW group, a single global gradient clip, and the encoder kept active for the full schedule alongside the transformer. Integrated into a full V-M-C pipeline with AdaLN-conditioned transformers and PPO-in-dreams, the system runs in real time at 30 FPS on Geometry Dash. SLS is in principle applicable to any model that predicts over a discrete latent space with a coordinate metric. A natural extension is to make the kernel bandwidth $\sigma$ learnable and jointly optimized with the WM, moving SLS from a fixed prior to a fully end-to-end objective; care must be taken to prevent degenerate solutions in which the learned targets collapse to uniform smoothing.

## Acknowledgments and Disclosure of Funding

We thank Maël Planchot for contributing gameplay data. The NVIDIA A100 GPU used for all primary training runs was provided by MesoNet (CRIANN, Rouen) through the Representation Learning course at INSA Rouen Normandy. Geometry Dash is developed by RobTop Games.

## References

[1] David Ha and Jürgen Schmidhuber. World models. In *Neural Information Processing Systems*, 2018.

[2] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023.

[3] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025.

[4] Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations*, 2024.

[5] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the Inception architecture. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2818–2826, 2016.

[6] Jost Tobias Springenberg, Alexey Dosovitskiy, Thomas Brox, and Martin Riedmiller. Striving for simplicity: The all convolutional net. In *International Conference on Learning Representations (Workshop)*, 2015.

[7] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *International Conference on Computer Vision*, 2023.

[8] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorber, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning*, 2024.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[10] Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models. *arXiv preprint arXiv:2509.24527*, 2025.

[11] Maxime Burchi and Radu Timofte. TWISTER: Learning transformer-based world models with contrastive predictive coding. In *International Conference on Learning Representations*, 2025.

[12] Mohammad Hassan Vali, Tom Bäckström, and Arno Solin. iFSQ: Improving FSQ for image generation with 1 line of code. In *International Conference on Learning Representations*, 2026.

[13] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[14] Tongzhou Wang and Phillip Isola. Understanding contrastive representation learning through alignment and uniformity on the hypersphere. In *International Conference on Machine Learning*, pages 9929–9939, 2020.

[15] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *AAAI Conference on Artificial Intelligence*, 2018.

[16] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *International Conference on Computer Vision*, pages 2980–2988, 2017.

[17] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

## NeurIPS Paper Checklist

1. **Claims**

   Answer: [Yes]

   Justification: The abstract and introduction state our two contributions (SLS and the real-time V-M-C pipeline) and scope them to a single game environment.

2. **Limitations**

   Answer: [Yes]

   Justification: Section 6 discusses single-environment evaluation, binary action space, and calibration transferability.

3. **Theory assumptions and proofs**

   Answer: [N/A]

   Justification: The paper does not include theoretical results. SLS is an empirical method.

4. **Experimental result reproducibility**

   Answer: [Yes]

   Justification: Section 4 provides full training details. Code will be released.

5. **Open access to data and code**

   Answer: [Yes]

   Justification: Code and dataset are publicly available at `https://github.com/Tariolle/sls-wm`. Data collection procedure is described in Section 4.

6. **Experimental setting/details**

   Answer: [Yes]

   Justification: All hyperparameters, optimizers, schedules, and data augmentation are specified in Section 4.

7. **Experiment statistical significance**

   Answer: **[TODO]**

   Justification: **[TODO]**

8. **Experiments compute resources**

   Answer: [Yes]

   Justification: All experiments run on a single NVIDIA A100 GPU. Training times are reported in Section 4.

9. **Code of ethics**

   Answer: [Yes]

   Justification: The research involves no human subjects, sensitive data, or dual-use concerns.

10. **Broader impacts**

    Answer: [N/A]

    Justification: This work applies to a single-player video game with no direct societal impact.

11. **Safeguards**

    Answer: [N/A]

    Justification: The released model is a small world model for a video game and poses no misuse risk.

12. **Licenses for existing assets**

    Answer: [Yes]

    Justification: All referenced works are properly cited. Geometry Dash is a commercial game by RobTop Games; gameplay data was collected from a legally purchased copy.

13. **New assets**

Answer: [Yes]

Justification: Code and model checkpoints will be released under an open-source license with documentation.

14. **Crowdsourcing and research with human subjects**

    Answer: [N/A]

    Justification: No crowdsourcing or human subjects research was conducted.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Answer: [N/A]

    Justification: No human subjects research was conducted.

16. **Declaration of LLM usage**

    Answer: [N/A]

    Justification: LLMs were not used as a component of the core methodology.